An Algorithm on Thin Walls

Marco Busetti

October 11, 2026

Introduction

This mini-report presents some visual results obtained in thin wall identification simulations for various geometries, along with the algorithms responsible for the method.

Some clarifications:

- **White edges:** outline of the outer polygon;
- **Yellow edges:** outlines of the polygon's holes;
- **Green edges:** medial axis of the geometry;
- **Colored-area polygons:** polygons that represent the thin wall regions (the polygons).

Spurious Branch Filtering

The first step in identifying thin walls is filtering the spurious branches of the straight skeleton of the polygon of the evaluated slice.

The spurious branch filtering problem is critical for thin wall analysis because it allows the removal of noise and original branches of the straight skeleton that are not part of the analysis for identifying thin walls and that do not correspond to the expected mechanical outcome of printing the part.

An algorithm that handles the filtering of spurious branches of a part through a topological and geometric procedure is proposed below.

Note that the spurious branch filtering problem may have different outputs depending on the user's intent. For a given part, the user may be interested in keeping or removing certain regions that can be considered spurious branches. This requires the introduction of a variable, the "Pruning Threshold L_{prune}", which corresponds to a multiple of the threshold established for classifying thin wall regions. That is:

$$L_{prune} = k \cdot T_{max},\ k \in \mathbb{R}_{\geq 0}$$

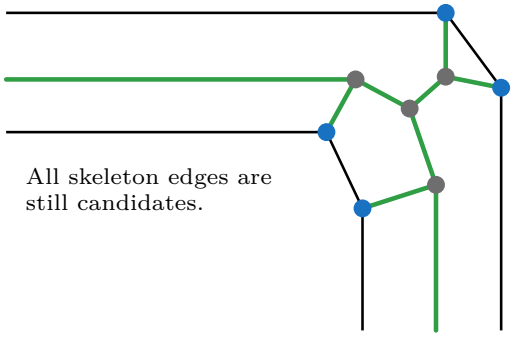
Where T_{max} represents the maximum diameter of the inscribed circle for the region to be classified as thin.

Algorithm 1: Filter Spurious Branches

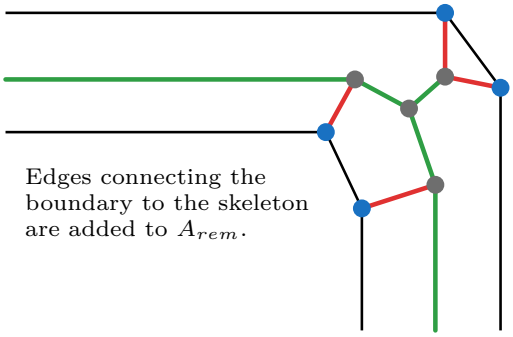
Input: Straight Skeleton S, Pruning Threshold L_{prune}.
Output: Set of edges to remove A_{rem}.

$A_{rem} \leftarrow \{e \in S.edges \mid e$ connects a boundary vertex to a skeleton vertex$\}$;
repeat
  $Items \leftarrow \emptyset$;
  for each *skeleton vertex* v **do**
    $g \leftarrow$ EffectiveDegree(v, A_{rem});
    if $g \geq 3$ **then**
      for each *edge* e *incident to* v *with* $e \notin A_{rem}$ **do**
        $C \leftarrow$ path that starts at v along e and continues through vertices of effective degree 2;
        if C *ends at a leaf* ***and*** *every vertex of* C *is at a distance less than* L_{prune} *from* v **then**
          $Items \leftarrow Items \cup \{(v, C, d_{max}, \text{false})\}$;
    if $g = 1$ **then**
      $C \leftarrow$ path that starts at leaf v and continues through vertices of effective degree 2;
      if C *ends at another leaf* ***and*** *every vertex of* C *is at a distance less than* L_{prune} *from* v **then**
        $Items \leftarrow Items \cup \{(v, C, d_{max}, \text{true})\}$;
  SortAscending$(Items, d_{max})$;
  $changed \leftarrow$ false;
  for each *item* $(v, C, d_{max}, island) \in Items$ **do**
    if $C \cap A_{rem} = \emptyset$ **then**
      if *island* ***or*** EffectiveDegree$(v, A_{rem} \cup C) \geq 1$ **then**
        $A_{rem} \leftarrow A_{rem} \cup C$;
        $changed \leftarrow$ true;
until $changed = false$;
return A_{rem};

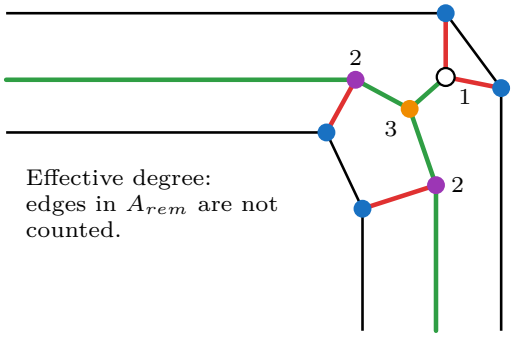
The idea of the algorithm above can be visualized with the following region of a given polygon:



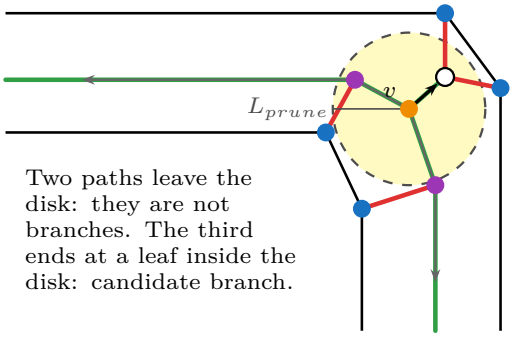
(a) Straight skeleton S.



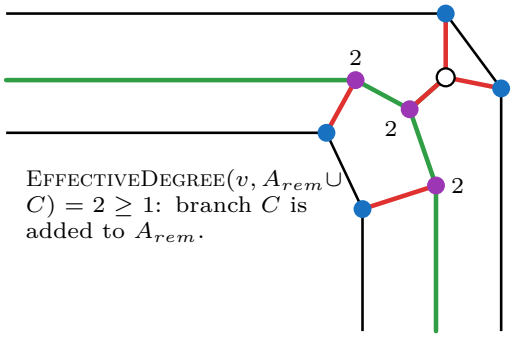
(b) Initial A_{rem}.



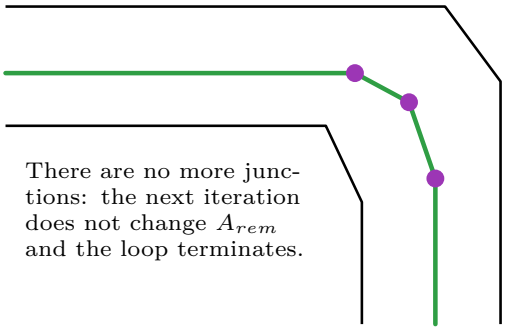
(c) Effective degrees of the vertices.



(d) Disk of radius L_{prune} centered at the junction v.



(e) Pruning of the spurious branch.



(f) Final skeleton $S' = S \setminus A_{rem}$.

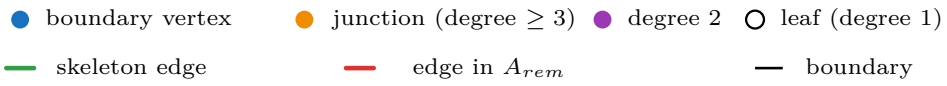
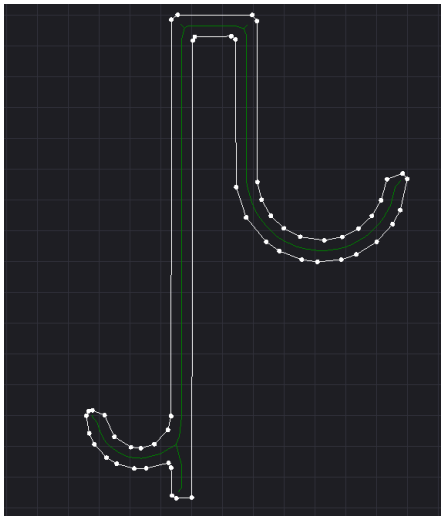
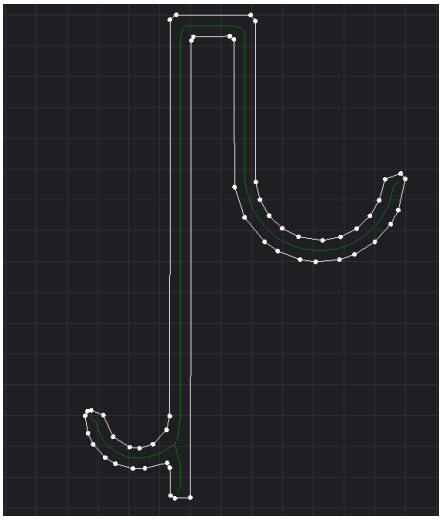


Figure 1: Step-by-step execution of Algorithm 1 on a thin wall corner. The cut in (e) only occurs because the junction v keeps an effective degree ≥ 1 after the removal.

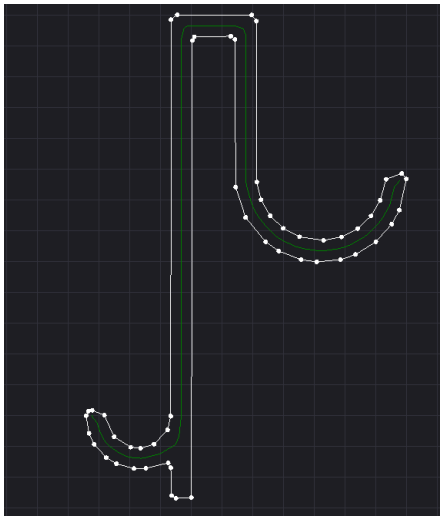
Below, some results of the filtered medial axis are presented for certain slices, with different values of L_{prune} for a given constant value of T_{max}.



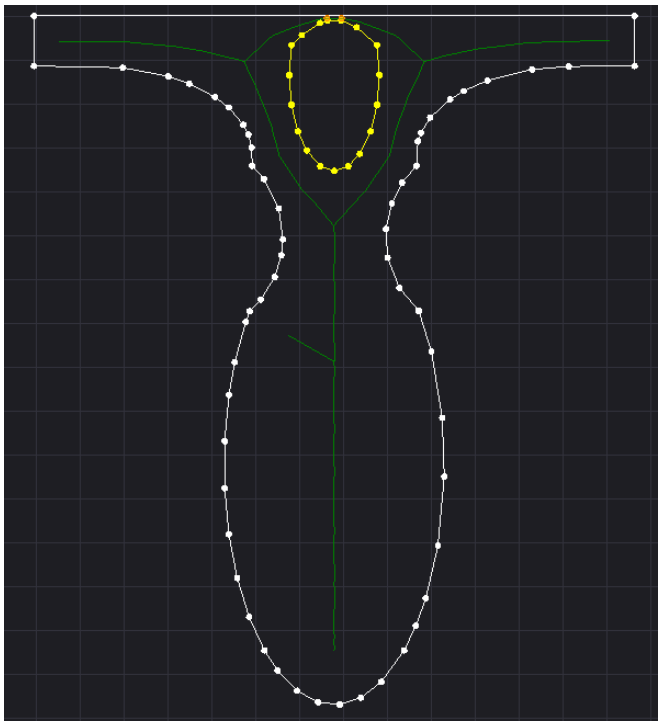
(a) Hook geometry with $L_{prune} = 0 \cdot T_{max}$



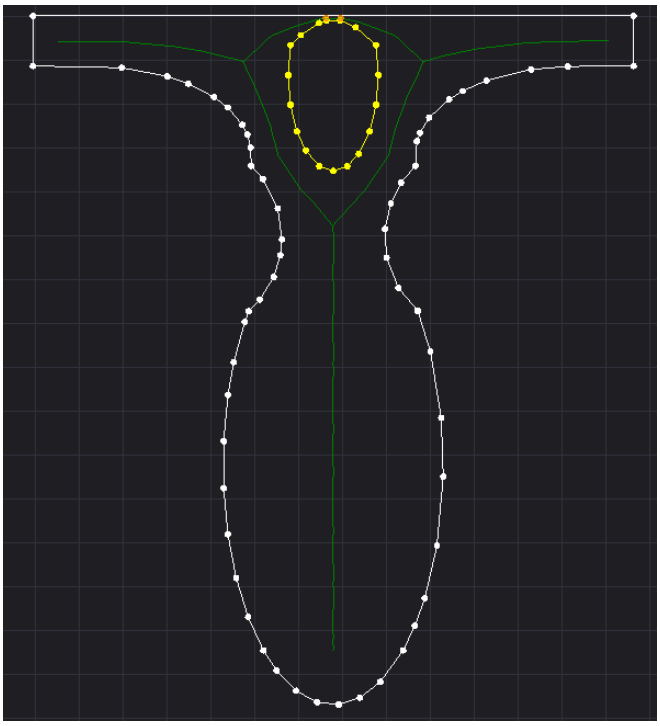
(b) Hook geometry with $L_{prune} = 1 \cdot T_{max}$



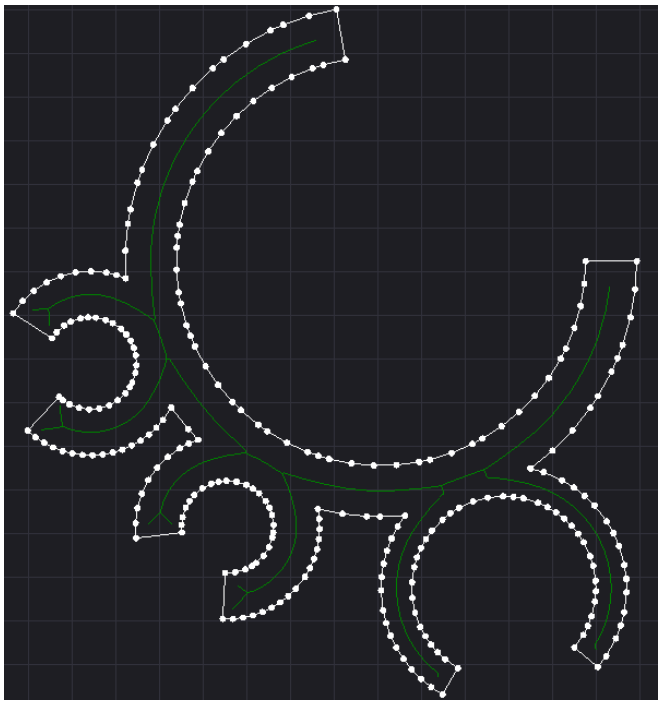
(c) Hook geometry with $L_{prune} = 5 \cdot T_{max}$



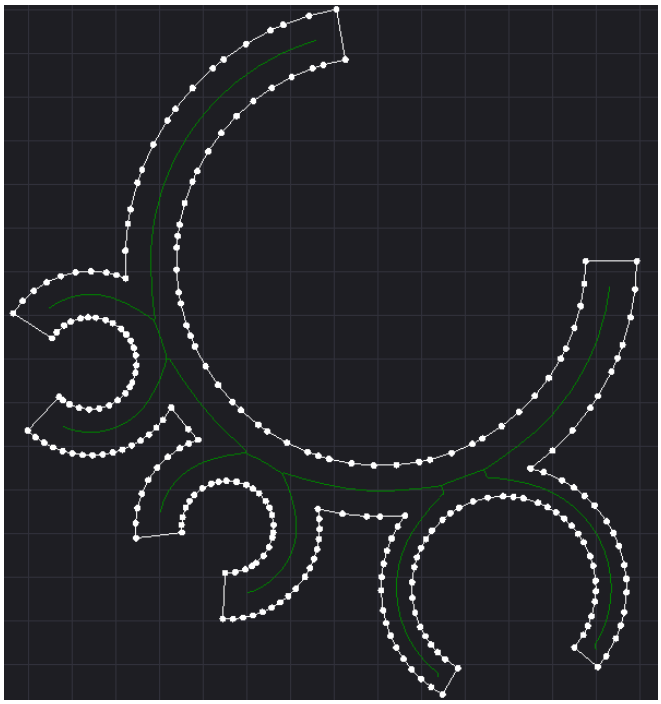
(a) Coat rack geometry with $L_{prune} = 0 \cdot T_{max}$



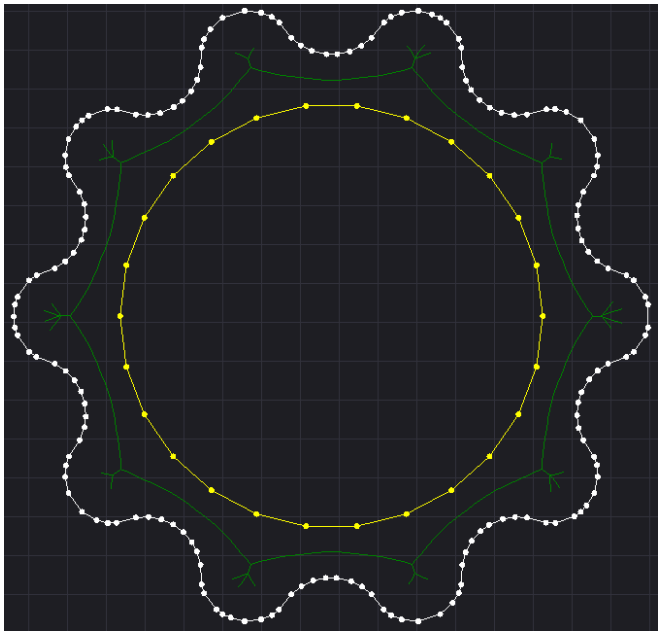
(b) Coat rack geometry with $L_{prune} = 10 \cdot T_{max}$



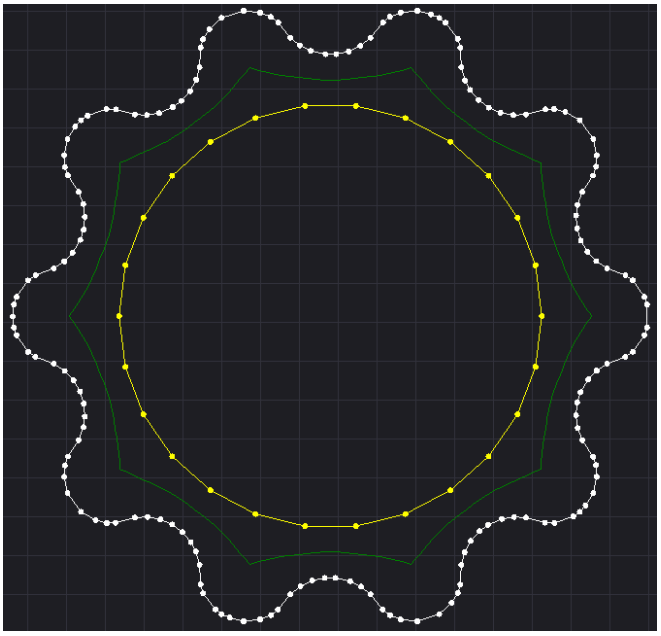
(a) Clamp geometry with $L_{prune} = 0 \cdot T_{max}$



(b) Clamp geometry with $L_{prune} = 5 \cdot T_{max}$



(a) Gear geometry with $L_{prune} = 0 \cdot T_{max}$



(b) Gear geometry with $L_{prune} = 3 \cdot T_{max}$

Thin Wall Identification

After the spurious branch filtering step, we can proceed to the thin wall identification method and the construction of the respective thin wall polygons. The algorithm that handles this task is proposed below:

Algorithm 1: Identify Thin Walls

Input: Polygon P, Holes H, Maximum Thickness T_{max}, Pruning Factor f_{prune}.
Output: List of polygons with holes $\mathcal{U}_{walls}$.

$\mathcal{L}_{chains} \leftarrow \emptyset$;
$S \leftarrow \text{BuildStraightSkeleton}(P, H)$;
$L_{prune} \leftarrow f_{prune} \cdot T_{max}$;
$A_{rem} \leftarrow \text{FilterSpuriousBranches}(S, L_{prune})$;
$E_{medial} \leftarrow \text{FilterMedialAxis}(S, A_{rem})$;
$Threshold \leftarrow T_{max}/2.0$;
/* Find the parametric bounds (t_0, t_1) of the thin segments */
$ThinSegments \leftarrow \emptyset$;
for each *half-edge* $he \in E_{medial}$ **do**
    $v_{src} \leftarrow he.origin, \quad v_{tgt} \leftarrow he.target$;
    $t_{src} \leftarrow v_{src}.time, \quad t_{tgt} \leftarrow v_{tgt}.time$;
    if $t_{src} < Threshold$ ***or*** $t_{tgt} < Threshold$ **then**
        $[t_0, t_1] \leftarrow \text{ComputeThresholdBounds}(t_{src}, t_{tgt}, Threshold)$;
        if $|t_1 - t_0| > \varepsilon$ **then**
            $ThinSegments[he] \leftarrow (he, t_0, t_1)$;
            $ThinSegments[he.opposite] \leftarrow (he.opposite, 1 - t_1, 1 - t_0)$;
/* Build continuous chains of thin walls */
$V_{visited} \leftarrow \emptyset$;
for each *half-edge* $he_{base} \in E_{medial}$ **do**
    if $he_{base} \in V_{visited}$ ***or*** $he_{base} \notin ThinSegments$ **then**
        continue;
    $Chain \leftarrow \text{StartChain}(ThinSegments[he_{base}])$;
    $\text{Mark}(V_{visited}, he_{base})$;
    /* Grow forward: continues while the segment extends to the end of the edge, the target vertex has degree 2 in S, and the next segment is thin and unvisited */
    $Chain \leftarrow \text{GrowChainForward}(Chain, ThinSegments, V_{visited})$;
    /* Grow backward: analogous, starting from the beginning of the first segment */
    $Chain \leftarrow \text{GrowChainBackward}(Chain, ThinSegments, V_{visited})$;
    $\Omega \leftarrow \text{BuildChainPolygon}(Chain)$;
    if $\Omega \neq \emptyset$ **then**
        $\mathcal{L}_{chains} \leftarrow \mathcal{L}_{chains} \cup \{\Omega\}$;
/* Unification of the thin walls */
$\mathcal{U}_{walls} \leftarrow \text{ExactPolygonUnion}(\mathcal{L}_{chains})$;
return $\mathcal{U}_{walls}$;

Algorithm 2: Build Chain Polygon

Input: Chain of thin segments $Chain = (c_1, \ldots, c_n)$, where $c_i = (he_i, t_0^i, t_1^i)$.
Output: Thin wall polygon Ω, or $\emptyset$ if the construction fails.

```
/* Chain endpoints on the medial axis                                    */
```
$Q_{start} \leftarrow$ StartPoint(c_1);
$Q_{end} \leftarrow$ EndPoint(c_n);
```
/* Projections onto the boundary edges that define the faces at the endpoints */
```
$Pl_{start} \leftarrow$ ProjectOntoBoundary(Q_{start}, he_1);
$Pr_{start} \leftarrow$ ProjectOntoBoundary$(Q_{start}, he_1.opposite)$;
$Pl_{end} \leftarrow$ ProjectOntoBoundary(Q_{end}, he_n);
$Pr_{end} \leftarrow$ ProjectOntoBoundary$(Q_{end}, he_n.opposite)$;
if *any projection fails* **then**
    return $\emptyset$;
$BoundaryPoints \leftarrow \emptyset$;
```
/* Initial cap and left side (forward)                                   */
```
Add$(BoundaryPoints, Q_{start})$;
Add$(BoundaryPoints, Pl_{start})$;
for $i \leftarrow 1$ **to** $n - 1$ **do**
    AddSharedVertex$(he_i, he_{i+1}, BoundaryPoints)$;
```
/* Final cap                                                             */
```
Add$(BoundaryPoints, Pl_{end})$;
Add$(BoundaryPoints, Q_{end})$;
Add$(BoundaryPoints, Pr_{end})$;
```
/* Right side (backward)                                                 */
```
for $i \leftarrow n$ **to** 2 *(decreasing)* **do**
    AddSharedVertex$(he_i.opposite, he_{i-1}.opposite, BoundaryPoints)$;
Add$(BoundaryPoints, Pr_{start})$;
```
/* Build and validate the polygon                                        */
```
$\Omega \leftarrow$ CreatePolygon$(BoundaryPoints)$;
if $|\Omega| < 3$ ***or*** Ω *is not simple* **then**
    return $\emptyset$;
OrientCounterclockwise(Ω);
return Ω;

Below, some tests performed for a given fixed T_{max} and $L_{prune} = 2 \cdot T_{max}$ are presented.

Two visualizations of the thin wall polygons are presented:

- "Chain" visualization of polygons: this visualization results from the procedural stitching of the polygons during the iteration over the medial axis edges of the geometry. Each polygon "chain" is the stitching of all thin wall polygons of a region bounded by vertices of degree ≥ 3. The visual effect is several groups of polygons denoted by different colors;
- Unified Visualization: this visualization is the result of the union of all the polygon groups generated during the stitching. The visual effect is a single polygon group denoted by one color.

Cogwheel

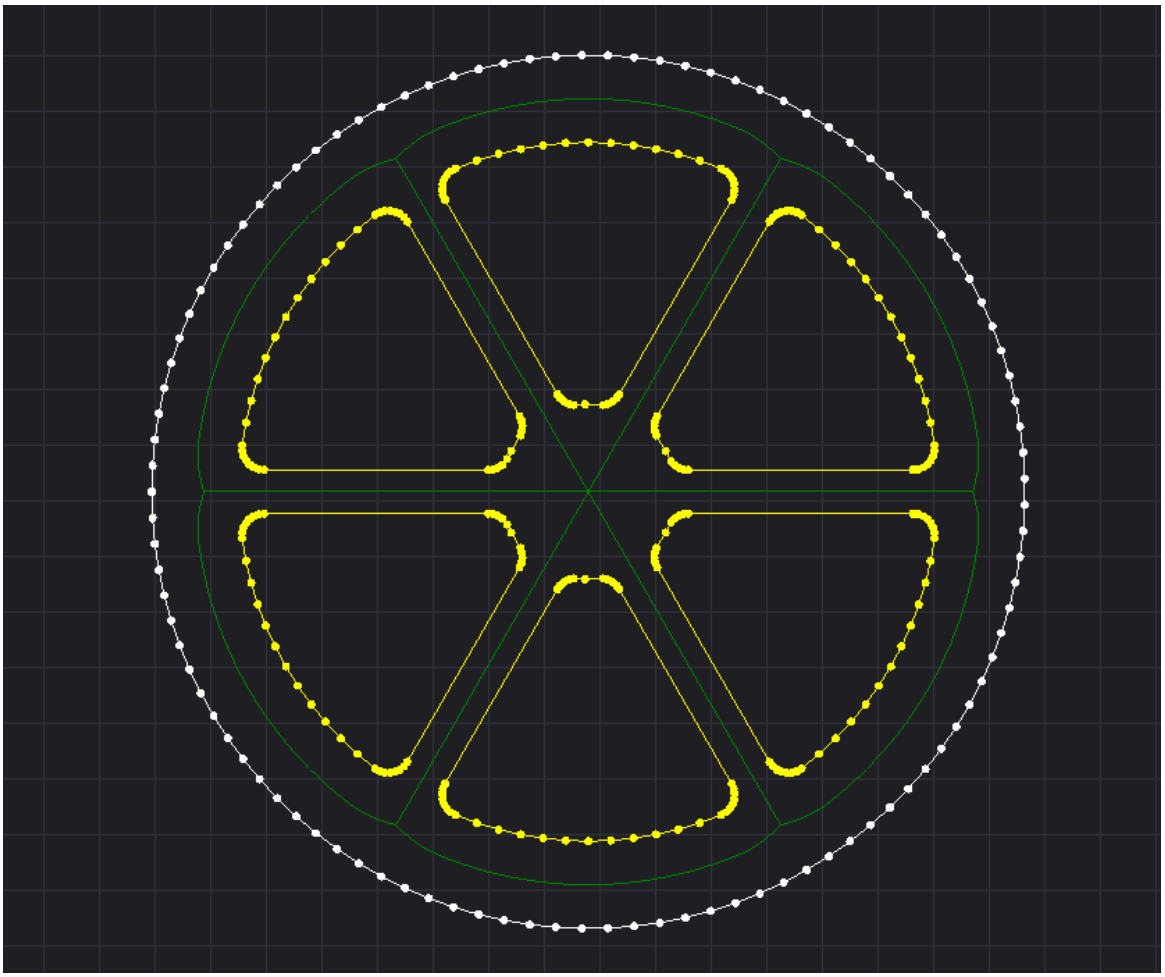
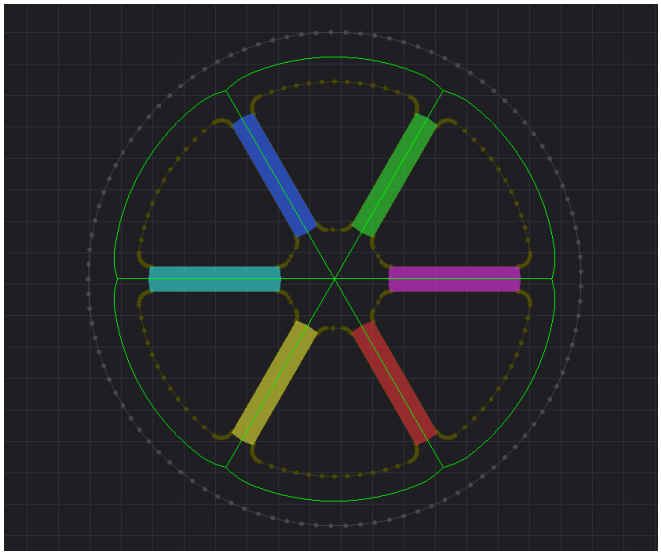
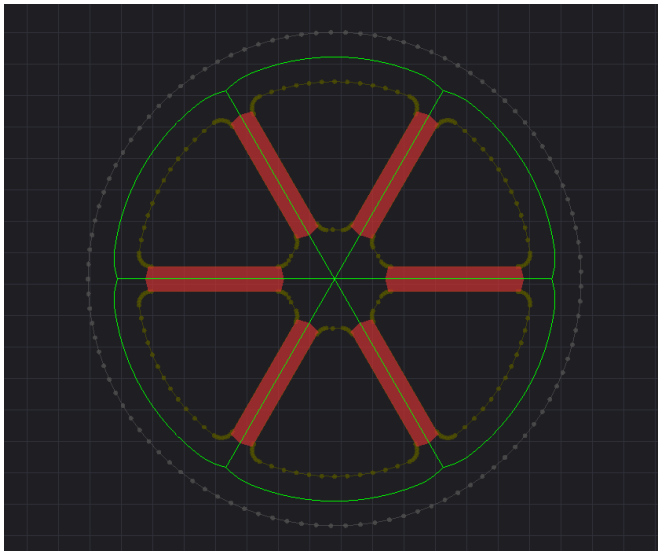


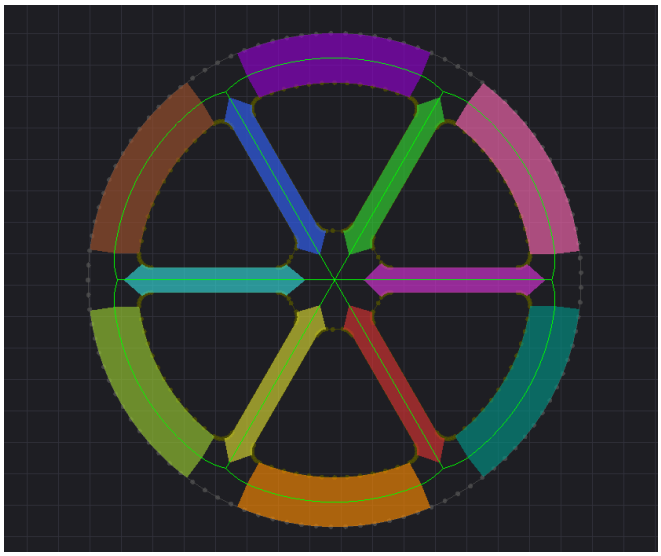
Figure 6: Polygon for the Cogwheel



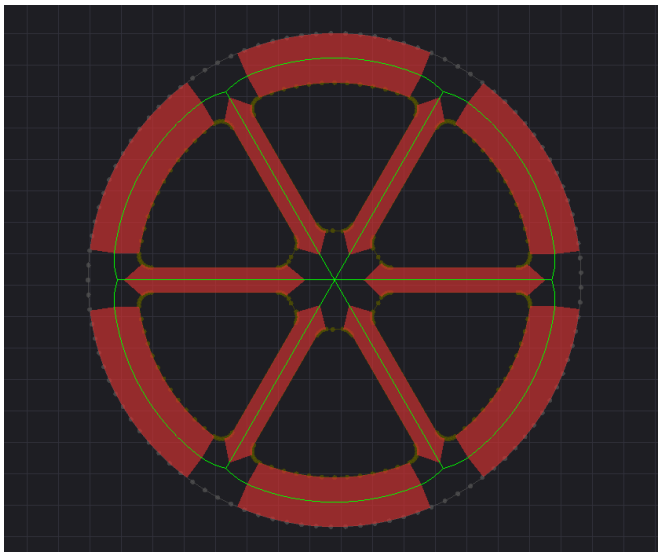
(a) Thin Walls in groups with *threshold* T_1



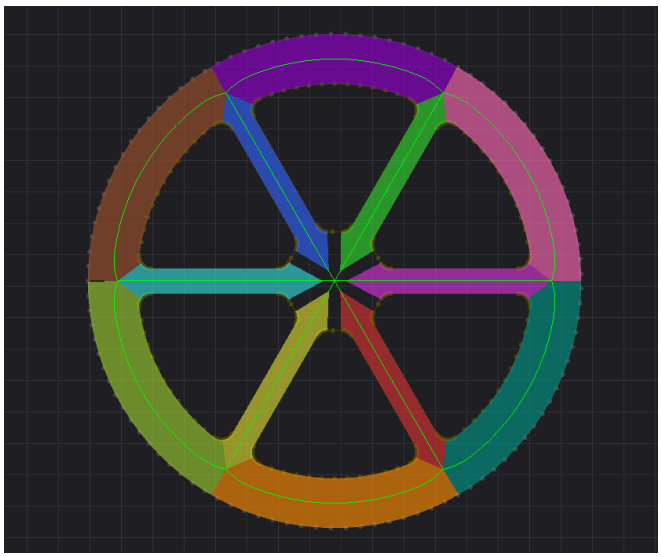
(b) Merged Thin Walls with *threshold* T_1



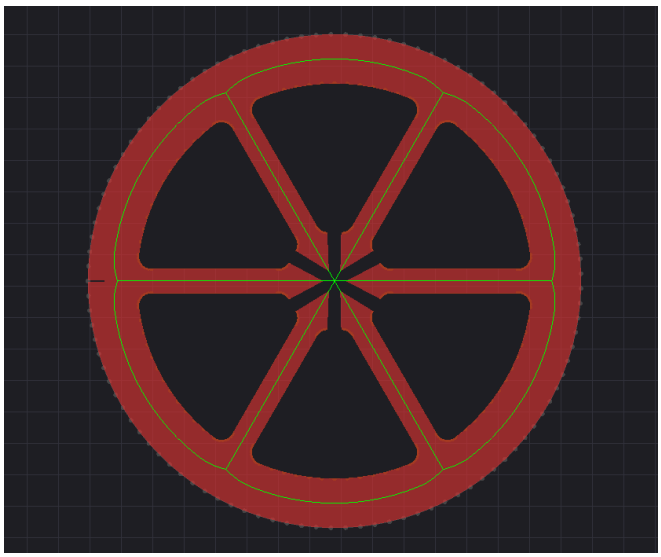
(c) Thin Walls in groups with *threshold* $T_2 > T_1$



(d) Merged Thin Walls with *threshold* $T_2 > T_1$



(e) Thin Walls in groups with *threshold* $T_3 > T_2$



(f) Merged Thin Walls with *threshold* $T_3 > T_2$

Figure 7: Different thin wall scenarios for the Cogwheel

Cogwheel with Hole

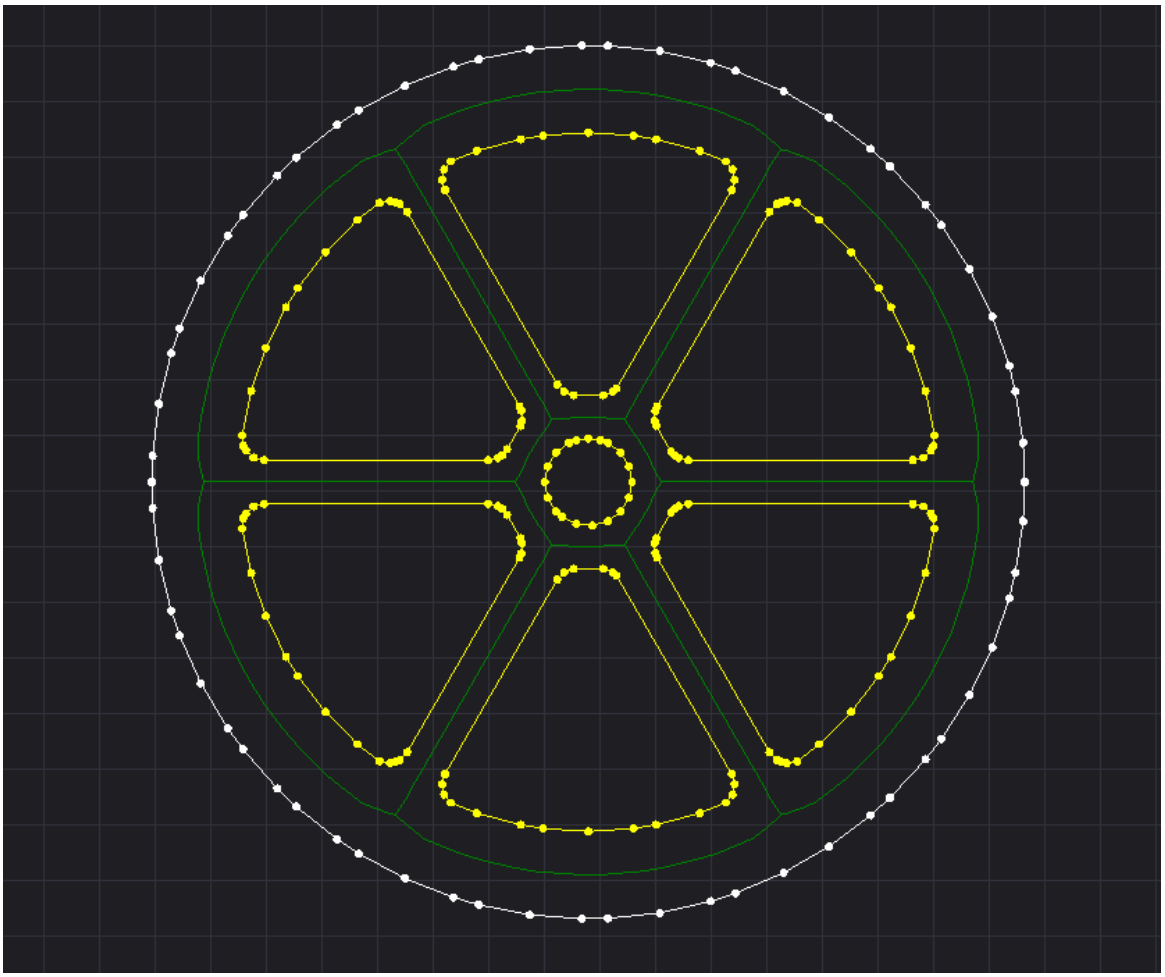
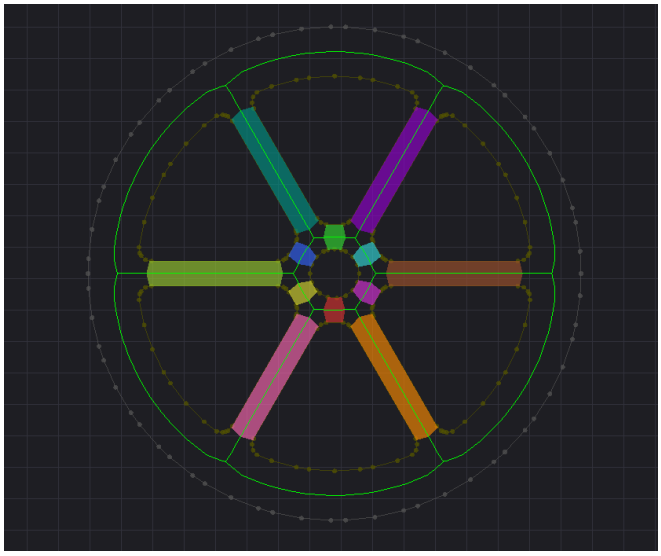
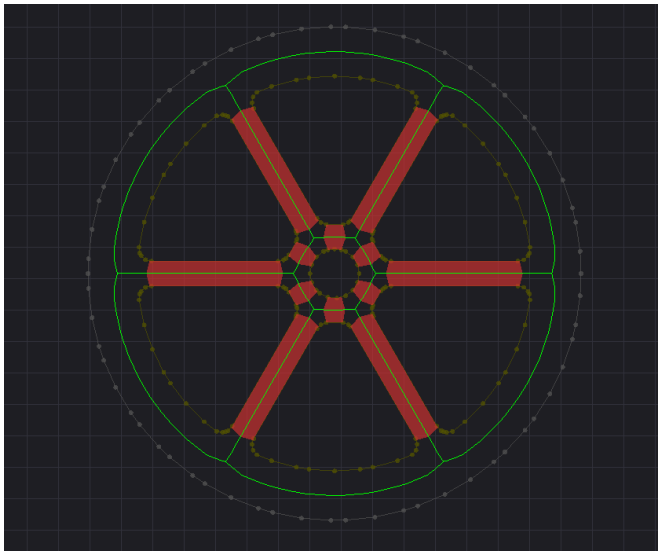


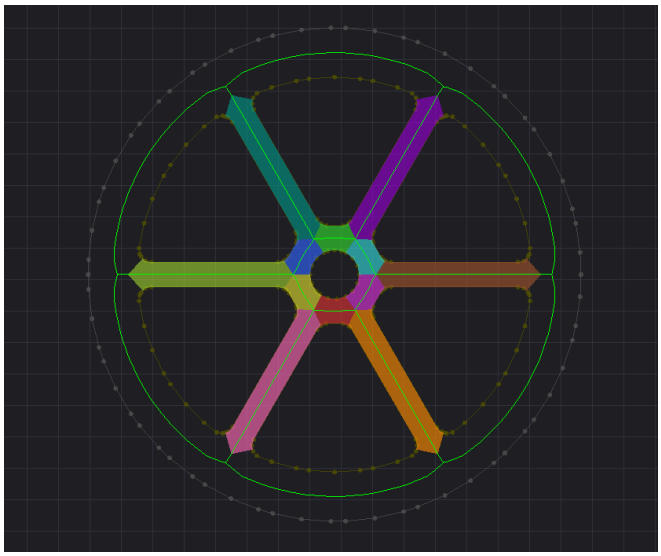
Figure 8: Polygon for the Cogwheel with Hole



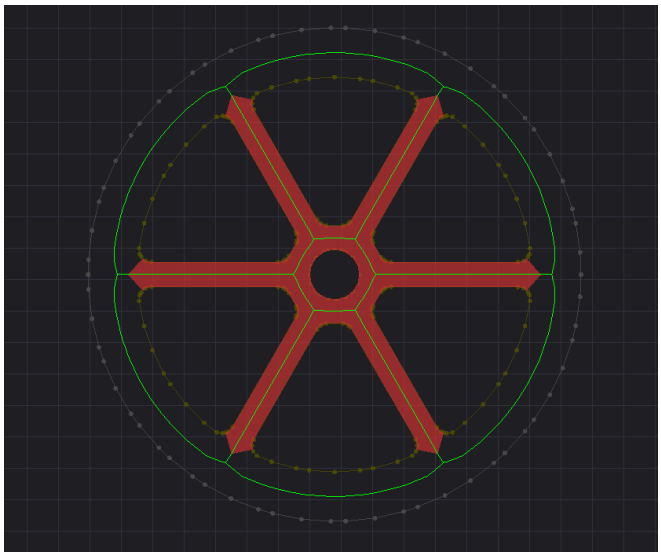
(a) Thin Walls in groups with *threshold* T_1



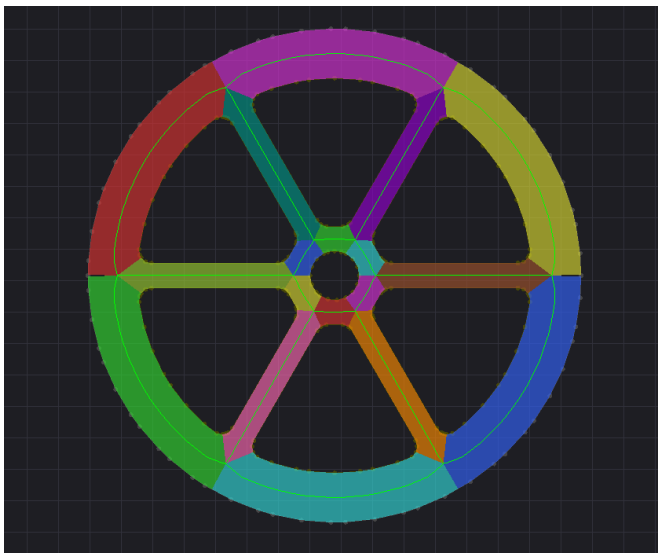
(b) Merged Thin Walls with *threshold* T_1



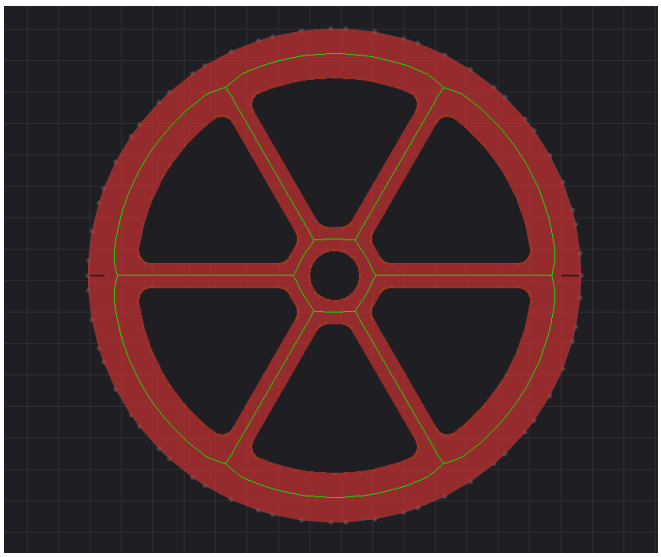
(c) Thin Walls in groups with *threshold* $T_2 > T_1$



(d) Merged Thin Walls with *threshold* $T_2 > T_1$



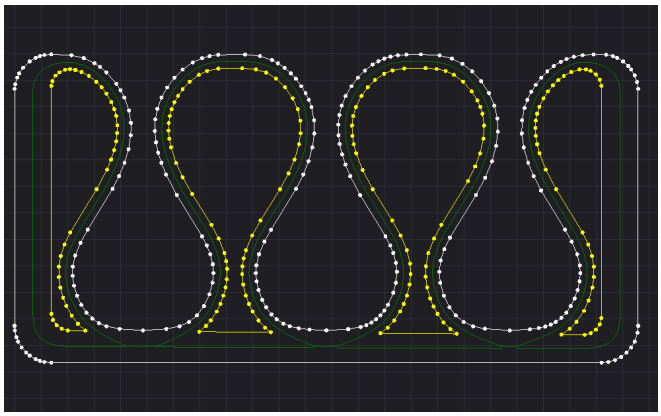
(e) Thin Walls in groups with *threshold* $T_3 > T_2$



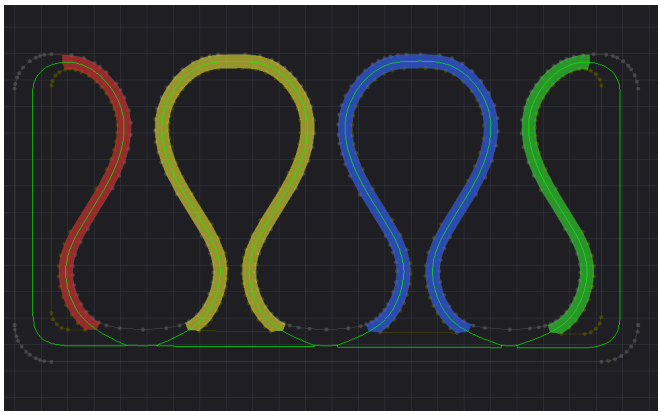
(f) Merged Thin Walls with *threshold* $T_3 > T_2$

Figure 9: Different thin wall scenarios for the Cogwheel with Hole

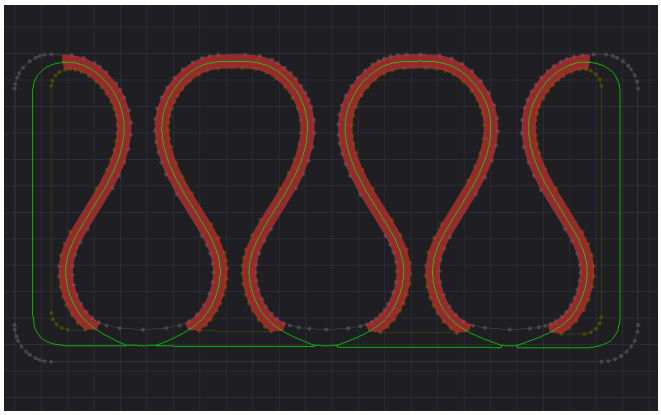
Miscellaneous Geometries



(a) Polygon for the Comb geometry

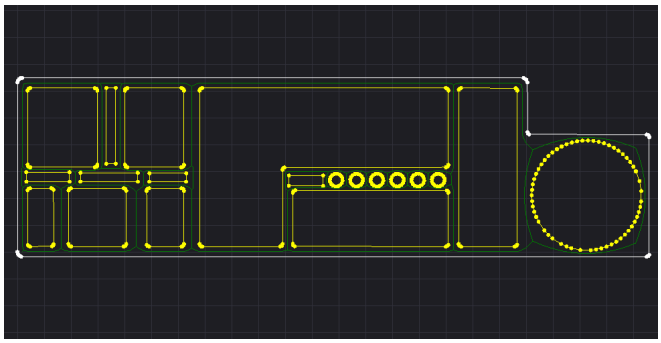


(b) Thin Walls in groups with *threshold* T

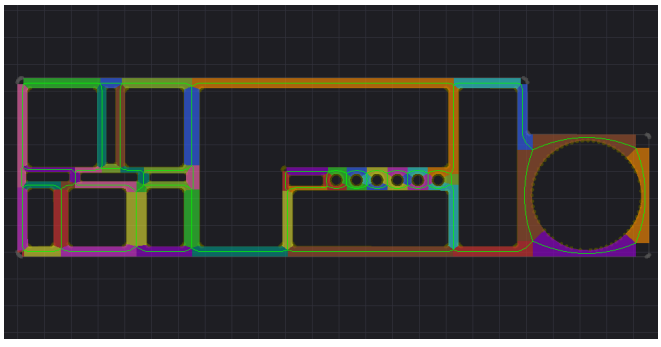


(c) Merged Thin Walls with *threshold* T

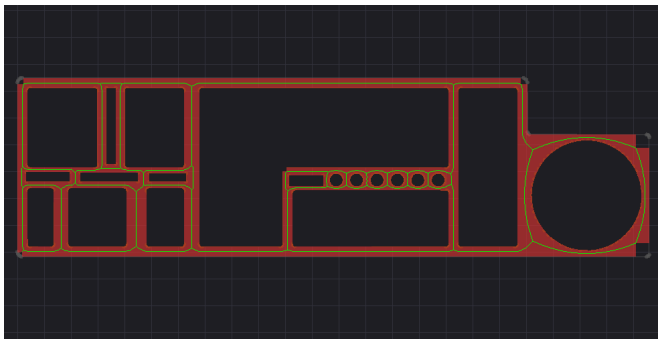
Figure 10: Thin Walls for Comb



(a) Polygon for the Maze geometry

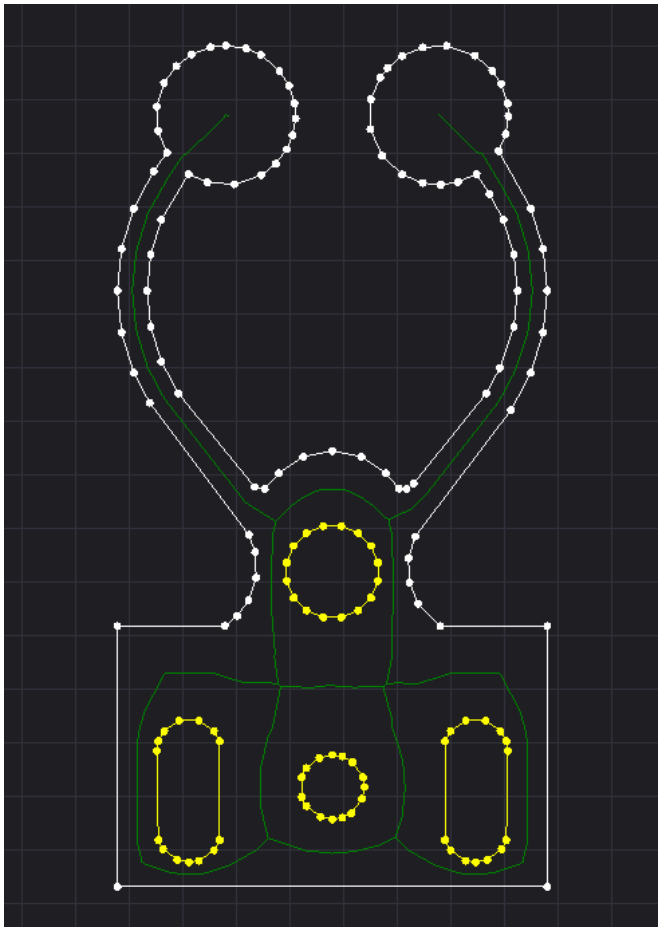


(b) Thin Walls in groups with *threshold* T

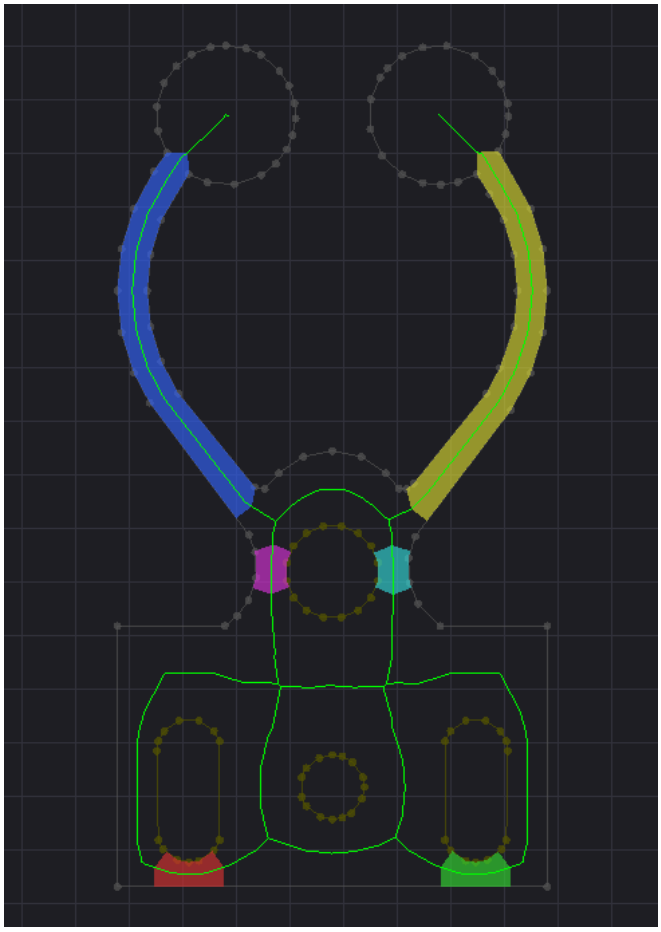


(c) Merged Thin Walls with *threshold* T

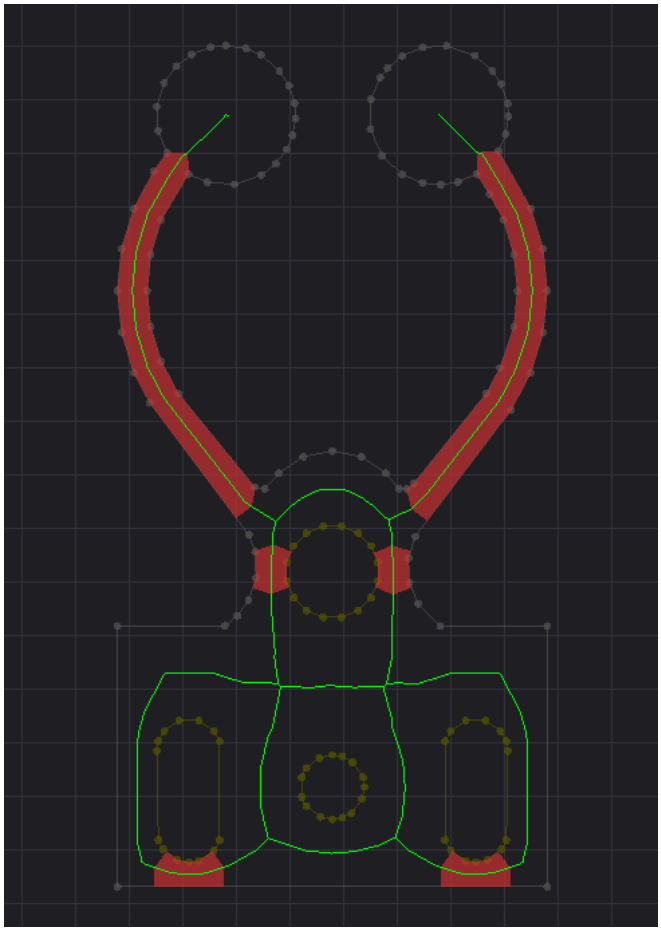
Figure 11: Thin Walls for Maze



(a) Polygon for the Tweezers geometry

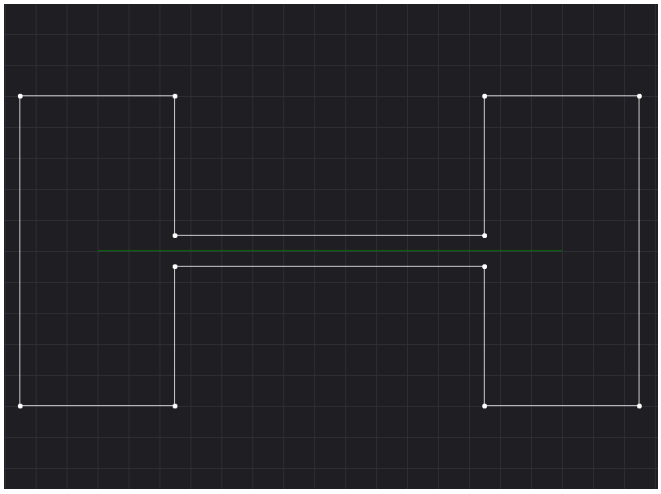


(b) Thin Walls in groups with *threshold T*

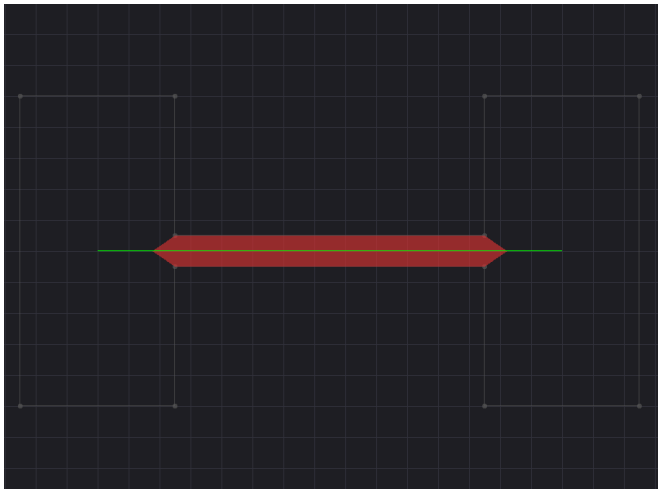


(c) Merged Thin Walls with *threshold T*

Figure 12: Thin Walls for Tweezers

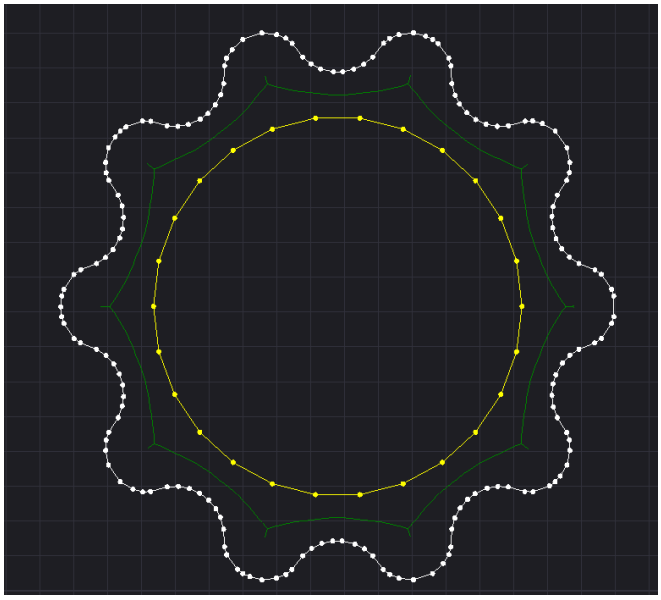


(a) Polygon for the dumbbell geometry

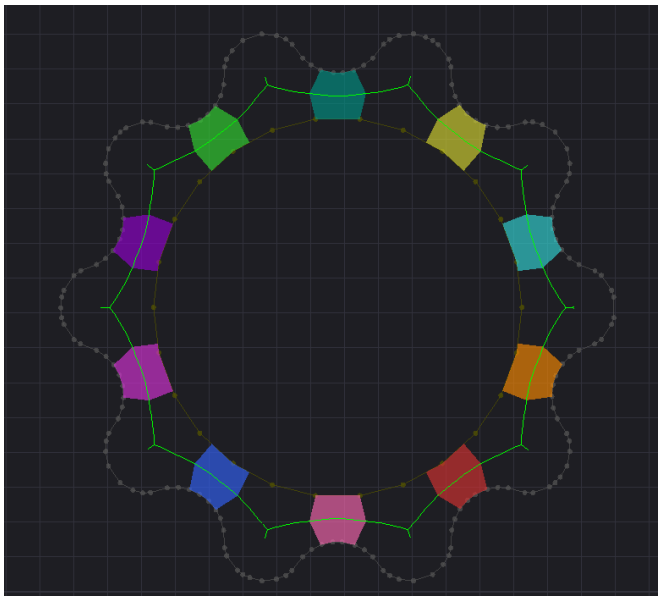


(b) Thin Wall polygon of the dumbbell with *threshold* T

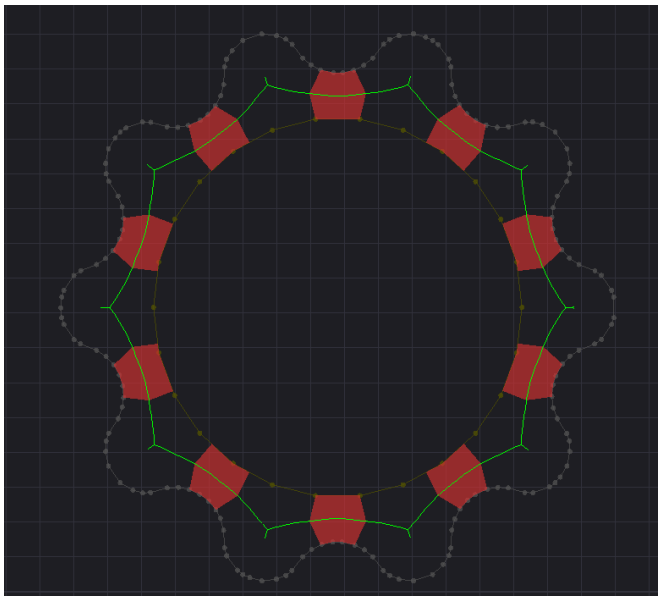
Figure 13: Thin Walls for dumbbell



(a) Polygon for the Gear geometry



(b) Thin Walls in groups with *threshold* T



(c) Merged Thin Walls with *threshold* T

Figure 14: Thin Walls for Gear